

01 UART Specification

- [SMC UART 16550 — Function Flow Reference](#)
 - [Contents](#)
 - [1. Overview](#)
 - [2. Big picture — entry points](#)
 - [3. TLM register access path](#)
 - [4. Register read side effects](#)
 - [5. Register write side effects](#)
 - [6. TX datapath](#)
 - [7. RX datapath](#)
 - [8. Interrupt and output evaluation](#)
 - [Interrupt priority\(`interrupt\_id\(\)` \)](#)
 - [DMA handshake inside `update\_outputs\(\)`](#)
 - [9. Modem flow](#)
 - [Loopback summary](#)
 - [10. SC_METHOD processes](#)
 - [11. Debug and test-bench API](#)
 - [12. Function inventory](#)
 - [Constructor](#)
 - [TLM interface](#)
 - [SC_METHOD processes](#)
 - [Output hub](#)
 - [Register decode](#)
 - [Datapath](#)
 - [Modem and interrupt](#)
 - [Helpers](#)
 - [Debug API \(public\)](#)
 - [13. Key rule — single output driver](#)
 - [Revision history](#)

SMC UART 16550 — Function Flow Reference

Document: 04_UART_Function_Flow.md

Module: smc::uart (include/uart.h, src/uart.cpp)

Status: Reference — call graph and data-flow for the SystemC LT model

Companion docs: - 01_UART_Specification.md — externally-observable behaviour -

02_UART_LowLevel_Design.md — internal implementation design - 03_UART_Test_Plan.md — verification strategy

Contents

1. [Overview](#)
2. [Big picture — entry points](#)
3. [TLM register access path](#)
4. [Register read side effects](#)
5. [Register write side effects](#)
6. [TX datapath](#)
7. [RX datapath](#)
8. [Interrupt and output evaluation](#)
9. [Modem flow](#)
10. [SC_METHOD processes](#)
11. [Debug and test-bench API](#)
12. [Function inventory](#)
13. [Key rule — single output driver](#)

1. Overview

The UART model follows the SMC **single-driver discipline**:

- Every `sc_out` port is written only by `update_outputs()`.
- Every state-changing path calls `schedule_recompute()`, which notifies `recompute_event_` at `SC_ZERO_TIME`.
- `recompute_method()` (sensitive to that event) calls `update_outputs()` in the next delta cycle.

External triggers that can change state:

Source	Entry function
CPU / fabric (<code>reg_socket</code>)	<code>b_transport()</code> , <code>transport_dbg()</code>
Test bench	<code>inject_rx_char()</code>
Reset pin	<code>reset_proc()</code>
Modem inputs	<code>modem_method()</code>
RX timeout timer	<code>rx_timeout_method()</code>

2. Big picture — entry points

```

flowchart TB
    subgraph External["External triggers"]
        CPU["CPU / fabric<br/>reg_socket"]
        TB["Test bench<br/>inject_rx_char()"]
        RST["rst_n_i"]
        MOD["Modem pins<br/>cts_ni, dsr_ni, ri_ni, dcd_ni"]
        TMO["rx_timeout_event_<br/>(timer)"]
    end

    subgraph TLM["TLM layer"]
        BT["b_transport()"]
        TD["transport_dbg()"]
    end

    subgraph Processes["SC_METHOD processes"]
        RP["reset_proc()"]
        MM["modem_method()"]
        RM["recompute_method()"]
        RTM["rx_timeout_method()"]
    end

    subgraph Hub["Output hub"]
        SR["schedule_recompute()"]
        UO["update_outputs()"]
    end

    CPU --> BT
    CPU --> TD
    TB --> INJ["inject_rx_char()"]
    INJ --> DRC["deliver_rx_char()"]

    RST --> RP
    MOD --> MM
    TMO --> RTM

    BT --> RR["reg_read() / reg_write()"]
    TD --> DBG["dbg_reg() / reg_write()"]

    RP --> SR
    MM --> SR
    RTM --> SR
    RR --> SR
    DRC --> SR

    SR -->| "notify SC_ZERO_TIME" | RM
    RM --> UO

    UO --> OUT["tx_o, irq_o, rxrdy_o,<br/>txrdy_o, err_o,<br/>rts_no, dtr_no, ..."]

```

3. TLM register access path

```

flowchart LR
    subgraph Read["READ path"]
        BTR["b_transport<br/>(READ)"]
        RR["reg_read()"]
        BTR --> RR

        RR --> RBR["rbr_read()"]
        RR --> IIR["iir_read()"]
        RR --> LSR["lsr_read()"]
        RR --> MSR["msr_read()"]
        RR --> PLAIN["regs_.ier/lcr/mcr/<br/>scr/ecr/itr/dll/dlm"]
    end

    subgraph Write["WRITE path"]
        BTW["b_transport<br/>(WRITE)"]
        RW["reg_write()"]
        BTW --> RW

        RW --> THR["thr_write()"]
        RW --> FCR["fcr_write()"]
        RW --> MCRW["MCR write +<br/>detect_modem_deltas()"]
        RW --> PLAINW["IER/LCR/SCR/<br/>ECR/ITR/DLL/DLM"]
    end

    end

    RBR --> SR["schedule_recompute()"]
    IIR --> SR
    LSR --> SR
    MSR --> SR
    THR --> SR
    FCR --> SR
    MCRW --> SR
    PLAINW --> SR

```

Validation in **b_transport()** (before dispatch):

Check	Response
Command not READ/WRITE	TLM_COMMAND_ERROR_RESPONSE
data_length != 4	TLM_BURST_ERROR_RESPONSE
Unaligned or addr >= WINDOW_SIZE	TLM_ADDRESS_ERROR_RESPONSE
Decode miss inside window	TLM_ADDRESS_ERROR_RESPONSE
Success	TLM_OK_RESPONSE + annotate access_delay_ns

DMI is never granted (RBR/IIR/MSR reads have side effects).

4. Register read side effects

flowchart TD

```
RR["reg_read(offset)"]
```

```
RR -->|0x00 DLAB=0| RBR["rbr_read()"]
```

```
RR -->|0x00 DLAB=1| DLL["return regs_.dll"]
```

```
RR -->|0x04 DLAB=0| IER["return regs_.ier"]
```

```
RR -->|0x04 DLAB=1| DLM["return regs_.dlm"]
```

```
RR -->|0x08| IIR["iir_read()"]
```

```
RR -->|0x0C| LCR["return regs_.lcr"]
```

```
RR -->|0x10| MCR["return regs_.mcr"]
```

```
RR -->|0x14| LSR["lsr_read()"]
```

```
RR -->|0x18| MSR["msr_read()"]
```

```
RR -->|0x1C| SCR["return regs_.scr"]
```

```
RR -->|0x20| ECR["return regs_.ecr"]
```

```
RR -->|0x24| ITR["return regs_.itr"]
```

```
RBR --> RBR1["pop rx_fifo_"]
```

```
RBR --> RBR2["update_rx_front_status()"]
```

```
RBR --> RBR3["clear rx_timeout_pending_"]
```

```
IIR --> IIR1["interrupt_id()"]
```

```
IIR --> IIR2["if THRE int: clear thre_intr_"]
```

```
LSR --> LSR1["any_rx_error()"]
```

```
LSR --> LSR2["clear sticky OE/PE/FE/BI"]
```

```
MSR --> MSR1["compute_modem_levels()"]
```

```
MSR --> MSR2["clear delta bits DCTS/DDSR/TERI/DDCD"]
```

```
MSR --> MSR3["snapshot prev_cts_/dsr_/ri_/dcd_"]
```

Register	Read side effect
RBR (0x00, DLAB=0)	Pop RX FIFO; update LSR.DR; clear RX timeout
IIR (0x08)	Compose IIR; clear THRE interrupt if it was the reported source
LSR (0x14)	Return value then clear sticky OE/PE/FE/BI
MSR (0x18)	Return value then clear delta bits; latch current modem levels

5. Register write side effects

flowchart TD

```
RW["reg_write(offset, data)"]
```

```
RW -->|0x00 DLAB=0| THR["thr_write(byte)"]
```

```
RW -->|0x00 DLAB=1| DLLW["regs_.dll = b<br/>+ drain_tx() if enabled"]
```

```
RW -->|0x04 DLAB=0| IERW["regs_.ier = b<br/>maybe set thre_intr_"]
```

```
RW -->|0x04 DLAB=1| DLMW["regs_.dlm = b<br/>+ drain_tx() if enabled"]
```

```
RW -->|0x08| FCR["fcr_write()"]
```

```
RW -->|0x10| MCR["regs_.mcr + detect_modem_deltas()"]
```

```
RW -->|0x0C| LCR["regs_.lcr"]
```

```
RW -->|0x20| ECR["regs_.ecr + update_rx_trigger()"]
```

```
RW -->|0x24| ITR["regs_.itr"]
```

```
RW -->|0x14, 0x18| R0["read-only: ignore"]
```

```
THR --> THR1["push tx_fifo_"]
```

```
THR --> THR2["drain_tx() if tx_enabled()"]
```

```
THR --> THR3["clear THRE/TEMT, thre_intr_=false"]
```

```
FCR --> FCR1["enable/reset RX and TX FIFOs"]
```

```
FCR --> FCR2["update_rx_trigger()"]
```

```
FCR --> FCR3["set fifo_en_, dma_mode_"]
```

```
THR --> SR["schedule_recompute()"]
```

```
FCR --> SR
```

```
MCR --> SR
```

```
IERW --> SR
```

```
DLLW --> SR
```

```
DLMW --> SR
```

```
LCR --> SR
```

```
ECR --> SR
```

```
ITR --> SR
```

Register	Write side effect
THR (0x00, DLAB=0)	Push TX FIFO; functionally transmit (LT drain)
FCR (0x08)	FIFO enable/reset; DMA mode; RX trigger
MCR (0x10)	Modem outputs / loopback; recompute MSR deltas
IER (0x04)	Enable interrupts; may raise THRE if THR already empty
ECR (0x20)	Update 4-bit RX FIFO trigger decode

6. TX datapath

```

flowchart TD
    THR["thr_write(byte)"]
    THR --> PUSH["tx_fifo.push_back()"]
    THR --> DT["drain_tx()"]

    DT --> LOOP{"MCR.LOOP &&<br/>!MCR.LINE_LOOP?"}
    LOOP -->|yes| DRC["deliver_rx_char()<br/>(system loopback)"]
    LOOP -->|no| HIST["tx_history.push_back()<br/>(observable via dbg_tx_pop)"]

    DT --> LSR["regs.lsr |= THRE | TEMT"]
    DT --> THRE["thre_intr_ = true"]

    DRC --> DRC1["update_rx_front_status()"]
    DRC --> DRC2["rearm_rx_timeout()"]
    DRC --> SR["schedule_recompute()"]

```

Notes:

- TX is disabled when baud divisor {DLM,DLL} == 0 (tx_enabled() returns false).
- In the LT model the entire TX FIFO drains in one drain_tx() call.
- SET_BREAK in LCR forces tx_o low via update_outputs(), not via the serialiser.

7. RX datapath

```

flowchart TD
    subgraph Sources["RX character sources"]
        INJ["inject_rx_char()"]
        LOOP["drain_tx() system loopback"]
    end

    INJ --> DRC["deliver_rx_char()"]
    LOOP --> DRC

    DRC --> EN{"rx_enabled()?<br/>(divisor != 0)"}
    EN -->|no| STOP["return"]
    EN -->|yes| FULL{"rx_fifo_full?"}

    FULL -->|yes FIFO mode| OE["LSR.OE = 1, drop char"]
    FULL -->|yes non-FIFO| OW["overwrite front entry"]
    FULL -->|no| PUSH["rx_fifo.push_back()"]

    PUSH --> UFS["update_rx_front_status()"]
    OW --> UFS
    UFS --> RT["rearm_rx_timeout()"]
    RT --> SR["schedule_recompute()"]

    RBR["rbr_read()"] --> POP["pop rx_fifo_"]
    POP --> UFS2["update_rx_front_status()"]
    POP --> CLR["clear rx_timeout_pending_"]
    UFS2 --> SR

```

RX timeout path:

```

rearm_rx_timeout()
-> rx_timeout_event_.notify(rx_timeout_delay_)
rx_timeout_method() (when event fires, FIFO non-empty)
-> rx_timeout_pending_ = true
-> schedule_recompute()

```

8. Interrupt and output evaluation

`update_outputs()` is called only from `recompute_method()`.

```

flowchart TD
    U0["update_outputs()"]

    U0 --> MOD["compute modem outputs<br/>(normal / sys loop / line loop)"]
    U0 --> TX["tx_o: break / line loop / idle"]
    U0 --> INT["interrupt_id()"]
    U0 --> DMA["DMA Mode 0 or Mode 1 FSM<br/>rxrdy_o, txrdy_o"]
    U0 --> ERR["err_o = LSR errors OR<br/>any_rx_error()"]

    INT --> IID["interrupt_id()"]
    IID --> CHK1["fifo_error + ITR/IER"]
    IID --> CHK2["line_status + ITR/IER"]
    IID --> CHK3["rx_timeout + ITR/IER"]
    IID --> CHK4["data_ready + ITR/IER"]
    IID --> CHK5["thre + ITR/IER"]
    IID --> CHK6["modem deltas + ITR/IER"]
    CHK6 --> IRQ["irq_o = (id != 0xFF)"]

    U0 --> DRIVE["write sc_out ports<br/>(with output cache)"]

```

Interrupt priority (`interrupt_id()`)

Fixed priority — first match wins:

Priority	IIR ID	Source	Condition
0 (highest)	0x7	FIFO Error	<code>any_rx_error()</code> in FIFO mode + IER.EFEI or ITR.TFEI
1	0x3	Line Status	LSR OE/PE/FE/BI + IER.ELSI or ITR.TLSI
2	0x6	RX Timeout	<code>rx_timeout_pending_</code> + IER.ERBFI or ITR.TRTI
3	0x2	Data Ready	RX count >= trigger (FIFO) or non-empty + IER.ERBFI or ITR.TRBFI
4	0x1	THRE	<code>thre_intr_</code> + IER.ETBEI or ITR.TTBEI
5 (lowest)	0x0	Modem Status	MSR delta bits + IER.EDSSI or ITR.TDSSI

ITR test bits **force** the corresponding source (bypass real condition and IER enable).

DMA handshake inside `update_outputs()`

Mode 0 (FCR.DMA_MODE_SELECT = 0):

```
rxrdy_o = (rx_fifo_ non-empty)
txrdy_o = (tx_fifo_ not full)
```

Mode 1 (FIFO enabled):

```
RX FSM: IDLE --(watermark or timeout)--> READY
         READY --(FIFO empty)--> IDLE
         rxrdy_o = (state == READY)

TX FSM: IDLE --(FIFO full)--> NOT_READY
         NOT_READY --(FIFO empty)--> IDLE
         txrdy_o = (state == IDLE)
```

9. Modem flow

```
flowchart TD
    subgraph Inputs["Modem input path"]
        PINS["cts_ni, dsr_ni,<br/>ri_ni, dcd_ni change"]
        PINS --> MM["modem_method()"]
        MM --> DMD["detect_modem_deltas()"]
        DMD --> CML["compute_modem_levels()"]
        DMD --> DELTA["set MSR DCTS/DDSR/<br/>DDCD/TERI sticky bits"]
        DELTA --> SR["schedule_recompute()"]
    end

    subgraph ReadMSR["MSR read path"]
        MSR["msr_read()"]
        MSR --> CML2["compute_modem_levels()"]
        MSR --> CLR["clear delta bits"]
        MSR --> SNAP["update prev_* snapshot"]
        CLR --> SR
    end

    CML --> MODE["loopback mode?"]
    MODE -->|normal| PIN["~cts_ni, ~dsr_ni, ..."]
    MODE -->|system loop| MCR["MCR RTS/DTR/OUT1/OUT2"]
    MODE -->|line loop| ZERO["all 0"]
```

Loopback summary

Mode	MCR	MSR bits 4–7	Modem outputs
Normal	LOOP=0 , LINE_LOOPBACK=0	From real pins	From MCR (~DTR , ~RTS , ...)
System loopback	LOOP=1	Reflect MCR control bits	Forced deasserted (high)
Line loopback	LINE_LOOPBACK=1	Forced 0	Follow modem inputs; tx_o follows rx_i

Line loopback takes precedence over system loopback.

10. SC_METHOD processes

Registered in the constructor with `dont_initialize()`:

```

flowchart LR
    subgraph Events["Sensitive to"]
        E1["rst_n_i"]
        E2["cts_ni, dsr_ni,<br/>ri_ni, dcd_ni"]
        E3["recompute_event_"]
        E4["rx_timeout_event_"]
    end

    E1 --> RP["reset_proc()"]
    E2 --> MM["modem_method()"]
    E3 --> RM["recompute_method()"]
    E4 --> RTM["rx_timeout_method()"]

    RP --> CLEAR["clear regs, FIFOs,<br/>FSM state, cancel timer"]
    RP --> SR["schedule_recompute()"]

    MM --> DMD["detect_modem_deltas()"]
    MM --> SR

    RTM --> T0["rx_timeout_pending_=true"]
    RTM --> SR

    RM --> UO["update_outputs()"]

```

Process	Trigger	Calls
<code>reset_proc()</code>	<code>rst_n_i</code> low	Clear all state; <code>compute_modem_levels()</code> baseline; <code>schedule_recompute()</code>
<code>modem_method()</code>	Any modem input change	<code>detect_modem_deltas()</code> ; <code>schedule_recompute()</code>
<code>recompute_method()</code>	<code>recompute_event_</code>	<code>update_outputs()</code> only
<code>rx_timeout_method()</code>	<code>rx_timeout_event_</code>	Set timeout flag; <code>schedule_recompute()</code>

11. Debug and test-bench API

Function	Calls	Side effects
<code>inject_rx_char()</code>	<code>deliver_rx_char()</code>	Full RX path + recompute
<code>dbg_tx_pop()</code>	reads <code>tx_history_</code>	None
<code>dbg_tx_count()</code>	reads <code>tx_fifo.size()</code>	None
<code>dbg_rx_count()</code>	reads <code>rx_fifo.size()</code>	None
<code>dbg_reg()</code>	<code>interrupt_id()</code> , <code>compute_modem_levels()</code> , <code>any_rx_error()</code>	No pop, no sticky clear
<code>transport_dbg()</code> READ	<code>dbg_reg()</code>	Same as above
<code>transport_dbg()</code> WRITE	<code>reg_write()</code>	Full write path
<code>dump_state()</code>	<code>interrupt_id()</code> , <code>tx_enabled()</code> , <code>any_interrupt()</code>	None

```

flowchart LR
    TD["transport_dbg()"]
    TD -->|READ| DBG["dbg_reg()"]
    TD -->|WRITE| RW["reg_write()"]

    INJ["inject_rx_char()"] --> DRC["deliver_rx_char()"]
    DRC --> SR["schedule_recompute()"]
    RW --> SR

```

12. Function inventory

Constructor

Function	Role
<code>uart()</code>	CCI setup; register TLM callbacks; register SC_METHOD processes

TLM interface

Function	Role
<code>b_transport()</code>	Main CPU register access; validate; <code>reg_read / reg_write</code> ; annotate delay
<code>transport_dbg()</code>	Debug access; read via <code>dbg_reg()</code> , write via <code>reg_write()</code>

SC_METHOD processes

Function	Role
<code>reset_proc()</code>	Clear state on active-low reset
<code>modem_method()</code>	Detect modem input deltas
<code>recompute_method()</code>	Sole caller of <code>update_outputs()</code>
<code>rx_timeout_method()</code>	Raise RX timeout interrupt

Output hub

Function	Role
<code>schedule_recompute()</code>	<code>recompute_event_.notify(SC_ZERO_TIME)</code>
<code>update_outputs()</code>	Drive all <code>sc_out</code> ports (irq, DMA, modem, tx, err)

Register decode

Function	Role
<code>reg_read()</code>	DLAB-aware read dispatch
<code>reg_write()</code>	DLAB-aware write dispatch
<code>rbr_read()</code>	Pop RX FIFO
<code>iir_read()</code>	Compose IIR; may clear THRE int
<code>lsr_read()</code>	Compose LSR; clear sticky errors
<code>msr_read()</code>	Compose MSR; clear deltas
<code>thr_write()</code>	Push and transmit byte
<code>fcr_write()</code>	FIFO control

Datapath

Function	Role
<code>drain_tx()</code>	LT transmit entire TX FIFO
<code>deliver_rx_char()</code>	Push RX character (inject or loopback)
<code>update_rx_front_status()</code>	Update LSR.DR/PE/FE/BI from FIFO front
<code>rearm_rx_timeout()</code>	Restart RX character-timeout timer
<code>update_rx_trigger()</code>	Decode {ECR,FCR} trigger level

Modem and interrupt

Function	Role
<code>compute_modem_levels()</code>	CTS/DSR/RI/DCD with loopback modes
<code>detect_modem_deltas()</code>	Set MSR sticky delta bits
<code>interrupt_id()</code>	Fixed-priority interrupt arbitration
<code>any_interrupt()</code>	<code>interrupt_id() != 0xFF</code>

Helpers

Function	Role
<code>tx_enabled()</code>	Divisor != 0
<code>rx_enabled()</code>	Same as <code>tx_enabled()</code>
<code>any_rx_error()</code>	Scan RX FIFO for error flags

Debug API (public)

Function	Role
<code>inject_rx_char()</code>	Back-door RX stimulus
<code>dbg_tx_pop()</code>	Pop transmitted character from history
<code>dbg_tx_count()</code> / <code>dbg_rx_count()</code>	FIFO occupancy
<code>dbg_reg()</code>	Side-effect-free register peek
<code>dump_state()</code>	Human-readable state dump

13. Key rule — single output driver

```
Any state change
  |
  v
schedule_recompute()
  |
  v
recompute_event_.notify(SC_ZERO_TIME)  <-- next delta cycle
  |
  v
recompute_method()
  |
  v
update_outputs()                        <-- ONLY writer of sc_out ports
  |
  +--> tx_o, irq_o, rxrdy_o, txrdy_o, err_o
  +--> rts_no, dtr_no, out1_no, out2_no
```

All other functions update **internal state** (registers, FIFOs, flags) and defer pin/output updates to this path. This satisfies SystemC's single-driver rule and keeps output ordering deterministic.

Revision history

Revision	Date	Notes
0.1	2026-06-25	Initial function-flow reference for <code>smc::uart</code>